

Unidad 2: Gestión Lean Agile de Productos de Software

Esta unidad se enfoca en la evolución de las prácticas ágiles para gestionar el desarrollo de software, centrándose en la escalabilidad, la eficiencia del flujo de trabajo y la medición del rendimiento.

Framework para escalar SCRUM / Nexus

Mientras que Scrum está diseñado para un solo equipo, el escalado ágil aborda el desafío de coordinar múltiples equipos Scrum (usualmente de 3 a 9) que trabajan sobre un **mismo Product Backlog** para construir un **único producto**. El principal desafío que resuelve es la gestión de **dependencias** (técnicas, de arquitectura o de dominio) que surgen entre los equipos.

Nexus es el framework oficial de Scrum.org para este fin. No es un proceso nuevo, sino una "capa" de integración que se añade sobre los equipos Scrum existentes.

- **Roles Clave:** Introduce el **Equipo de Integración de Nexus (Nexus Integration Team - NIT)**. Este equipo no es de gestión, sino de servicio y coordinación. Está compuesto típicamente por el **Product Owner** (que es el PO de todo el Nexus), un **Scrum Master** (que facilita el Nexus) y **miembros de los equipos de desarrollo** (representantes con las habilidades técnicas para resolver la integración). Su responsabilidad principal es asegurar que un **Incremento de Producto Integrado** y "Hecho" (Done) se produzca al final de cada Sprint, gestionando las dependencias que impiden esto.
- **Artefactos Clave:**
 - **Product Backlog:** Existe un único Product Backlog para todos los equipos. El trabajo de **Refinamiento (Refinement)** se vuelve crucial para identificar y visualizar las dependencias entre los ítems.
 - **Nexus Sprint Backlog:** Es la composición de los Sprint Backlogs de todos los equipos del Nexus. Se vuelve transparente para que todos puedan ver el trabajo de los demás y las dependencias identificadas.
 - **Incremento Integrado:** Es la suma de todo el trabajo "Hecho" completado por *todos* los equipos al final del Sprint. Debe estar verdaderamente integrado y ser potencialmente desplegable.
- **Eventos Coordinados:** Se mantienen los eventos de Scrum, pero se les añade una capa de coordinación:
 - **Nexus Sprint Planning:** El NIT presenta el Product Backlog y las dependencias. Luego, los equipos planifican por separado, pero vuelven a reunirse para ajustar y consolidar un único **Nexus Sprint Goal**.
 - **Nexus Daily Scrum:** Representantes de cada equipo se reúnen *primero* para identificar problemas de integración del día anterior y nuevos problemas para el día actual. Esta información se lleva *luego* a los Daily Scrums de cada equipo.
 - **Nexus Sprint Review:** Es un *único* evento (no uno por equipo) donde se presenta el Incremento Integrado a los stakeholders.
 - **Nexus Sprint Retrospective:** Ocurre en tres partes: (1) Representantes se reúnen para identificar problemas a escala; (2) Cada equipo tiene su retrospectiva; (3) Los representantes se reúnen de nuevo para consolidar los aprendizajes y generar acciones de mejora para el Nexus.

Filosofía Lean - Método Kanban en el contexto del desarrollo de software

La **Filosofía Lean** (Lean Thinking) se origina en el Sistema de Producción de Toyota (TPS) y su objetivo principal es maximizar el valor entregado al cliente **minimizando el desperdicio** ("muda").

- **Los 7 Desperdicios (Muda) del Software:**

1. **Trabajo Parcialmente Hecho (Inventario):** Código no integrado, software no desplegado que no aporta valor.
2. **Funcionalidades Extra (Sobreproducción):** Desarrollar "gold-plating" o características que el cliente no pidió.
3. **Re-aprendizaje (Procesamiento Extra):** Conocimiento no compartido, silos de información, mala documentación.
4. **Transferencias (Handoffs):** Pasar trabajo de un rol a otro (ej. de Desarrollador a QA), lo que genera esperas y pérdida de contexto.
5. **Cambio de Contexto (Movimiento):** Desarrolladores trabajando en múltiples tareas a la vez, perdiendo eficiencia.
6. **Retrasos (Esperas):** Tiempos muertos esperando aprobaciones, builds, o resolución de dependencias.
7. **Defectos (Re-trabajo):** Errores que obligan a rehacer el trabajo.

Kanban es un *método* (no una metodología prescriptiva) que aplica los principios Lean. Se centra en la gestión del flujo de trabajo y la mejora incremental (Kaizen). A diferencia de Scrum, no prescribe roles, iteraciones (sprints) ni reuniones fijas.

- **Principios y Prácticas Clave:**

1. **Visualizar el flujo de trabajo:** Usando un tablero Kanban que mapea las etapas reales del proceso (ej. "Backlog", "Análisis", "En Desarrollo", "Pruebas", "Desplegado").
2. **Limitar el Trabajo en Progreso (WIP):** Este es el principio central. Se establece un límite numérico explícito a la cantidad de tareas que pueden estar en una columna (o en el sistema). Esto crea un **sistema "Pull"** (solo se "tira" de una nueva tarea cuando hay capacidad disponible), lo cual reduce el cambio de contexto, expone cuellos de botella y acorta el tiempo de entrega.
3. **Gestionar el Flujo:** Medir y optimizar el movimiento de las tareas (tarjetas) por el tablero. El objetivo es que el flujo sea rápido y predecible.
4. **Hacer las políticas explícitas:** Todos entienden las reglas (ej. "Definición de Hecho" para cada columna, cómo se prioriza, cómo se manejan los bloqueos).
5. **Implementar Ciclos de Retroalimentación (Cadencias):** Kanban fomenta reuniones según sea necesario, como la *Replenishment Meeting* (para rellenar el backlog) o la *Service Delivery Review* (para analizar el rendimiento del flujo).
6. **Mejorar Colaborativamente, Evolucionar Experimentando (Kaizen):** El tablero y las políticas se ven como un modelo científico que se mejora continuamente.

Métricas en los 3 enfoques

- **Tradicional (Predictivo/Cascada):** El objetivo es la **conformidad con el plan** inicial.
 - **EVM (Gestión del Valor Ganado):** Mide el desempeño del proyecto comparando el trabajo planificado (PV - Planned Value), el costo real (AC - Actual Cost) y el valor del trabajo completado (EV - Earned Value).
 - **CPI (Cost Performance Index) = EV / AC .** (Si < 1 , está por encima del presupuesto).
 - **SPI (Schedule Performance Index) = EV / PV .** (Si < 1 , está retrasado).
 - **Densidad de Defectos:** (Nro. de defectos / KLOC o Puntos de Función).
- **Agile (Adaptativo/Scrum):** El objetivo es la **entrega de valor y la predictibilidad** del equipo.
 - **Velocidad (Velocity):** Promedio de "Story Points" que un equipo completa por Sprint. Se usa *solo* para **planificación y forecasting** (predecir cuántos Sprints tomará un release). **No es una métrica de productividad** ni debe usarse para comparar equipos.
 - **Gráfico Burndown (Sprint):** Muestra el trabajo *restante* en un Sprint. Es una herramienta para el equipo, para ver si alcanzarán el Sprint Goal.
 - **Gráfico Burnup (Release):** Muestra el trabajo *completado* acumulado contra el *alcance total*. Es una herramienta para el Product Owner, ya que visualiza el progreso y el impacto del "scope creep" (cambios de alcance).
- **Lean (Flujo/Kanban):** El objetivo es la **eficiencia y predictibilidad del flujo**.
 - **Lead Time:** Tiempo total desde que un cliente *pide* una funcionalidad hasta que la *recibe* (perspectiva del cliente).
 - **Cycle Time:** Tiempo total desde que se *empieza a trabajar* en una tarea hasta que se *termina* (perspectiva del equipo). El objetivo es reducirlo.
 - **Throughput:** Cantidad de ítems entregados por unidad de tiempo (ej. "5 tareas por semana").
 - **Ley de Little (Little's Law):** Fórmula matemática clave que vincula estas métricas: $WIP = Throughput \times Cycle Time$. (WIP Promedio = Throughput Promedio $\times$ Cycle Time Promedio). Demuestra que para un throughput constante, si se incrementa el WIP, inevitablemente se incrementará el Cycle Time.